Geometric Shielding and Lagrangian Constraints are Complementary: An Empirical Study in Safe Reinforcement Learning

Samuele Pesacane

https://github.com/samuelepesacane/Safe-Reinforcement-Learning-for-Robotic-Manipulation/

Abstract

Safe reinforcement learning requires balancing task performance against safety constraints, yet how different safety mechanisms interact when combined remains an open question. We present a systematic empirical comparison of four reinforcement learning algorithms, PPO, SAC, RCPO, and Lagrangian PPO (LagPPO), each trained with and without a geometric keepout shield on the SafetyPointPush1-v0 benchmark. Our 4×2 factorial experiment across three random seeds reveals that safety mechanisms are not interchangeable and exhibit distinct trade-offs. RCPO achieves the lowest constraint violation rate throughout training but at significant cost to task return. SAC converges fastest in return but incurs high safety cost without explicit constraints. Most notably, we find evidence that geometric shielding and Lagrangian constraints are complementary: adding a shield measurably reduces the growth of the Lagrange multiplier in LagPPO, suggesting the two mechanisms operate at different timescales. Unconstrained PPO relies most heavily on the shield, exhibiting intervention rates substantially higher than constrained algorithms. No algorithm achieves zero violations within one million training steps, highlighting the difficulty of full constraint satisfaction under compute constraints. Our results suggest that safety mechanisms should be treated as composable components with distinct trade-offs, and that the appropriate choice depends on whether the priority is constraint satisfaction, task performance, or both.

1 Introduction

Deploying reinforcement learning agents in the real world requires more than optimizing for task performance. A robot operating in a shared workspace must avoid collisions with humans and objects. An autonomous system navigating a constrained environment must respect physical boundaries. In these settings, an agent that learns to complete its task without regard for safety is not a useful agent. Safe reinforcement learning addresses this problem by incorporating constraints into the learning process, ensuring that the agent pursues its objective within defined safety boundaries.

Two broad families of methods have emerged for enforcing safety during training. Constraint-based methods, such as Lagrangian relaxation and reward penalty approaches, modify the optimization objective to penalize unsafe behavior. Shield-based methods intercept the agent's actions before they reach the environment, projecting unsafe actions onto a safe set defined by geometric or probabilistic models of the hazard structure. Both families have been studied extensively in isolation, and each has demonstrated advantages in specific settings. What remains less understood is how these mechanisms interact when combined: does adding a shield to a constrained algorithm improve safety further, or do the two mechanisms address the same problem redundantly?

This question is practically important. Real deployments often combine multiple safety layers, and a practitioner building a safe RL system needs to know whether those layers are additive, redundant, or potentially conflicting. Existing benchmarks and comparisons typically evaluate methods as alternatives rather than as composable components, leaving this interaction largely unmeasured.

We address this gap through a controlled empirical study. We train four algorithms, PPO, SAC, RCPO, and LagPPO, each with and without a geometric keepout shield, on the SafetyPointPush1-v0 benchmark from Safety-Gymnasium. Our 4×2 factorial design across three random seeds allows us to isolate the effect of shielding within each algorithm family and compare behavior across the full set of conditions. We find that safety mechanisms are not interchangeable and exhibit distinct trade-offs that persist across one million training steps. Most notably, geometric shielding measurably reduces the growth of the Lagrange multiplier in LagPPO, providing empirical evidence that, to the best of our knowledge, these two mechanisms are complementary rather than redundant and operate at different timescales.

Our contributions are three-fold. First, we provide a systematic empirical comparison of four safe RL algorithms with and without geometric shielding, finding that RCPO achieves the lowest violation rate at the cost of near-zero task return, while unconstrained algorithms incur high cost but competitive return, and that no algorithm satisfies the cost budget within one million steps. Second, we report empirical evidence that geometric shielding and Lagrangian constraints operate at different timescales and are complementary when combined, with shielding measurably reducing Lagrange multiplier growth in LagPPO. Third, we release an open-source experimental framework supporting Safety-Gymnasium environments and custom robotic models, enabling reproducible benchmarking of safe RL methods under controlled conditions.

2 Related Work

Empirical comparisons of safe RL algorithms have grown more systematic following the introduction of standardized benchmarks. Ray et al. [2019] introduce Safety Gym and provide the first large-scale comparison of constrained RL methods, finding that Lagrangian-based methods perform competitively but do not consistently satisfy constraints within their training budgets. Ji et al. [2023b] extend this with OmniSafe, a unified infrastructure encompassing on-policy, off-policy, and model-based safe RL algorithms evaluated across the Safety-Gymnasium suite. While these works establish strong baselines, they evaluate safety mechanisms as alternatives rather than examining how mechanisms interact when combined.

Shield-based methods have been studied primarily as standalone safety mechanisms. Dalal et al. [2018] propose the safety layer, which corrects policy actions by solving an analytic projection at each step using a learned linear model of constraint dynamics. Their approach demonstrates that runtime action correction can prevent constraint violations during training without modifying the learning objective. Control barrier function methods [Ames et al., 2019] provide a formal alternative grounded in control theory, guaranteeing forward invariance of safe sets under continuous dynamics. Our geometric shield follows the same principle as Dalal et al. [2018] but replaces the learned dynamics model with explicit knowledge of hazard geometry, making it simpler and fully interpretable at the cost of requiring known environment structure.

The interaction between shielding and learning-based safety methods has received limited direct study. Most work treats the two families as alternatives. To the best of our knowledge, our work is the first to empirically measure the interaction between geometric shielding and Lagrangian constraints within a controlled factorial design, providing direct evidence that the two mechanisms are complementary rather than redundant.

3 Background

3.1 Safe Reinforcement Learning and Constrained MDPs

Standard reinforcement learning trains an agent to maximize expected cumulative reward through interaction with a Markov decision process (MDP). This formulation makes no distinction between desirable and undesirable states: the agent will exploit any behavior that increases return, including unsafe ones encountered during exploration. Safe reinforcement learning extends this framework by formalizing safety as an explicit constraint on the learning problem.

The standard formalism is the constrained Markov decision process (CMDP), introduced by Altman [1999] and adopted as the foundation for modern safe RL benchmarks by Ray et al. [2019]. A CMDP extends the standard MDP with an additional cost function and budget, giving the tuple $(\mathcal{S}, \mathcal{A}, P, r, c, \gamma, d)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition distribution, $r(s, a)$ is the reward function, $c(s, a) \geq 0$ is the cost function, $\gamma \in [0, 1)$ is the discount factor, and d is the maximum allowable expected cumulative cost. At each timestep the environment produces a reward r_t measuring task progress and a cost c_t measuring safety violations. The learning objective is to find a policy π that maximizes expected cumulative reward $J_R(\pi)$ subject to the constraint that expected cumulative cost $J_C(\pi)$ remains below d:

$$\max_{\pi} \; J_R(\pi) \quad \text{s.t.} \quad J_C(\pi) \leq d, \tag{1}$$

where

$$J_R(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \qquad J_C(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t c_t \right]$$

3.2 Constraint-Based Algorithms

LagPPO

LagPPO [Ray et al., 2019] applies Lagrangian relaxation to convert the constrained optimization in Equation 1 into an unconstrained saddle-point problem. A Lagrange multiplier $\lambda \geq 0$ is introduced as an adaptive penalty coefficient, and the policy maximizes the modified objective:

$$\max_{\pi} \min_{\lambda \geq 0} \; J_R(\pi) - \lambda\big(J_C(\pi) - d\big). \tag{2}$$

Intuitively, λ acts as a dynamic price for safety violations. When the policy exceeds the cost budget, λ increases, making violations more costly in the objective and pushing the policy toward safer behavior. When the policy satisfies the constraint, λ decreases and the algorithm prioritizes reward. The multiplier is updated after each policy rollout by:

$$\lambda \;\leftarrow\; \max\big(0,\; \lambda + \eta_\lambda \left(J_C(\pi) - d\right)\big), \tag{3}$$

where η_λ is the multiplier learning rate.

RCPO

RCPO [Tessler et al., 2018] is a multi-timescale approach to constrained policy optimization that incorporates safety as a penalty term directly into the reward function, updating the penalty coefficient on a slower timescale than the policy until the constraint is satisfied. The shaped reward at each step takes the form:

$$r'_t \;=\; r_t - \lambda \cdot c_t, \tag{4}$$

where λ is a penalty coefficient updated on a slower timescale than the policy gradient. In our implementation, we use a fixed penalty coefficient $\lambda_{\text{fixed}} = 1.0$ rather than the full adaptive multi-timescale update, making it closer to static reward shaping than to the original RCPO algorithm. This simplified baseline serves as a static penalty comparison point. In practice this means the penalty coefficient does not respond to constraint violations during training, making it more stable but less responsive than the dynamic Lagrangian update used by LagPPO. A key empirical difference is that our RCPO baseline treats safety as a fixed shaping term during any given policy update, while LagPPO enforces the constraint as an active optimization target throughout training.

3.3 Geometric Action Shielding

Shield-based methods intervene at execution time to prevent unsafe actions from reaching the environment. Dalal et al. [2018] introduce the safety layer concept, showing how to add a correction layer on top of a policy that analytically solves an action correction formulation at each state. Their approach learns a linear model of constraint dynamics from data. The geometric shield used in this work follows the same principle but replaces the learned dynamics model with explicit geometric knowledge of the environment: hazard zones are represented as circular keepout regions with known positions and radii, making the shield simpler and fully interpretable, at the cost of requiring known hazard geometry at deployment time.

When the policy proposes an action a_t, the shield predicts the agent's next position and checks whether it falls within any hazard region. If a violation is predicted, the unsafe component of the action is removed by projecting out the direction toward the hazard, producing a corrected action a_t^{safe} that keeps only the tangential or outward component of the original action.

This design creates a clean separation of timescales: the shield enforces instantaneous, step-level safety by construction, while constraint-based algorithms such as LagPPO enforce distributional safety over episodes through the accumulated cost signal. Whether these two mechanisms are complementary or redundant when combined is the central empirical question of this paper.

3.4 Unconstrained Baselines

We include two unconstrained algorithms as baselines. Proximal Policy Optimization [PPO; Schulman et al., 2017] is an on-policy policy gradient method that clips the policy update ratio

$$r_t(\theta) = \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$$

to prevent large policy changes, optimizing the clipped surrogate objective. Soft Actor-Critic [SAC; Haarnoja et al., 2018] is an off-policy actor-critic method that maximizes the entropy-regularized objective

$$J(\pi) = \mathbb{E}_\pi \left[\sum_t r_t + \alpha \, \mathcal{H}(\pi(\cdot \mid s_t)) \right]$$

making it sample-efficient and well-suited to continuous control. Neither algorithm includes explicit safety constraints. They are included to measure baseline task performance and to quantify how much the shield alone affects behavior in the absence of any constraint.

4 Experimental Setup

4.1 Environment

All experiments are conducted on SafetyPointPush1-v0 from the Safety-Gymnasium benchmark [Ji et al., 2023a]. The task requires a point robot to push a box to a goal position while navigating around circular hazard regions scattered throughout a planar arena. The environment produces two signals at each timestep: a reward measuring progress toward the goal, and a cost of 1.0 whenever the agent occupies a hazard region. The observation space is a flat vector encoding the agent's position and velocity, the box and goal positions, and pseudo-lidar readings indicating proximity to nearby hazards. The task creates a natural conflict between performance and safety, as direct paths to the goal may pass through hazard regions.

4.2 Algorithms

We evaluate four algorithms in a 4×2 factorial design. PPO [Schulman et al., 2017] and SAC [Haarnoja et al., 2018] serve as unconstrained baselines with no cost awareness. RCPO [Tessler et al., 2018] is implemented as a fixed-penalty baseline with penalty coefficient $\lambda_{\text{fixed}} = 1.0$, serving as a static penalty comparison point. LagPPO [Ray et al., 2019] enforces the cost constraint via an adaptive Lagrange multiplier updated after each rollout with learning rate $\eta_\lambda = 5 \times 10^{-4}$, and a per-step cost budget of $d = 0.05$. This budget corresponds to an average of one hazard step per 20 environment steps, representing a moderately tight constraint that requires the policy to actively avoid hazards while still permitting occasional exploration near boundaries. Each algorithm is trained with and without a geometric keepout shield, yielding eight experimental conditions.

4.3 Geometric Shield

The geometric shield intercepts policy actions before they reach the environment and corrects any action that would move the agent into a known hazard region. Hazard positions and radii are read directly from the environment at the start of each episode. At each step, the shield predicts the agent's next position given the proposed action and checks for hazard overlap. If a violation is predicted, the component of the action directed toward the hazard is removed, keeping only the tangential or outward component. This produces the closest safe action to the original proposal. Unlike learned safety layers [Dalal et al., 2018], the shield requires no training and operates purely from geometric knowledge of the environment.

4.4 Training Details

All conditions are trained for one million environment steps with four parallel environment workers. Policy networks are standard two-layer MLPs. Each condition is trained with three independent random seeds, giving 24 total runs. All experiments are conducted on a single NVIDIA RTX 3070 GPU. Full implementation details, hyperparameters, and reproduction commands are provided in Appendix A.

4.5 Evaluation Metrics

After training, each checkpoint is evaluated for 20 episodes on a fresh environment instance. No shield is applied during evaluation, so results reflect learned policy behavior without runtime safety correction. We report mean and standard deviation across three seeds for each condition. The following metrics are used throughout:

- **Average episodic return:** mean cumulative reward per episode, measuring task performance.
- **Average episodic cost:** mean cumulative safety cost per episode, where cost accumulates at 1.0 for each step inside a hazard region.
- **Violation rate:** fraction of evaluation episodes in which any safety cost was incurred.
- **CVaR cost ($\alpha = 0.1$):** Conditional Value at Risk at the 10% level, i.e., the mean cost of the worst 10% of episodes, measuring tail safety risk rather than average behavior.
- **Shield intervention rate:** fraction of training steps at which the shield modified the policy's proposed action, reported from training logs for shield-on conditions only.
- **Lagrange multiplier trajectory λ:** reported from training logs for LagPPO conditions only, measuring whether the cost constraint is binding during learning.

5 Results

We structure our results around three questions. First, how do the four algorithms compare in task performance and safety? Second, does adding a geometric shield consistently improve safety? Third, how do the shield and the Lagrangian constraint interact in LagPPO? Table 1 summarizes evaluation results across all eight conditions. Training dynamics are shown in Figures 1, 2, 3, and 4.

5.1 Task Performance and Safety Trade-offs

Figure 1 shows episode return during training across all eight conditions. PPO, SAC, and LagPPO all converge to episodic returns between 1.0 and 1.5 by one million training steps, regardless of shield condition. SAC learns fastest, reaching a stable return of approximately 1.0 by 200k steps, consistent with its off-policy sample efficiency. The sharp dip in SAC return around 150k steps reflects its entropy-driven exploration phase and recovers normally. RCPO is the clear outlier: both shield conditions remain near zero return throughout training, indicating that the fixed penalty coefficient suppresses task-seeking behavior almost entirely. This reflects the fundamental trade-off of static penalty methods, where the penalty is strong enough to discourage unsafe behavior but also strong enough to discourage the task itself.

Table 1: Evaluation results (mean $\pm$ std across 3 seeds, 20 evaluation episodes each). CVaR is reported at $\alpha = 0.1$.

Condition	**Avg Return**	**Avg Cost**	**Violation Rate**	**CVaR**
PPO shield-off	1.10 ± 0.47	53.4 ± 21.2	0.32 ± 0.12	375.8
PPO shield-on	0.68 ± 0.21	48.3 ± 45.5	0.25 ± 0.16	323.6
SAC shield-off	1.07 ± 0.11	21.2 ± 17.0	0.13 ± 0.00	200.5
SAC shield-on	0.95 ± 0.12	57.3 ± 56.0	0.15 ± 0.14	416.0
RCPO shield-off	-0.90 ± 0.44	3.6 ± 3.8	0.08 ± 0.10	26.1
RCPO shield-on	-0.26 ± 0.44	5.6 ± 8.1	0.03 ± 0.05	56.3
LagPPO shield-off	0.78 ± 0.23	49.8 ± 27.9	0.30 ± 0.15	314.8
LagPPO shield-on	0.84 ± 0.57	34.7 ± 28.6	0.22 ± 0.10	285.8

Figure 1: Episode return during training (mean ± std across 3 seeds).

Evaluation results in Table 1 confirm this picture. PPO and SAC achieve mean returns of approximately 1.07 to 1.10, while RCPO achieves -0.90 and -0.26 for shield-off and shield-on respectively. LagPPO falls in the middle at 0.78 and 0.84. On safety, RCPO achieves the lowest average cost across all conditions (3.6 ± 3.8 for shield-off, 5.6 ± 8.1 for shield-on), confirming that the static penalty suppresses violations even at the cost of task performance. PPO and LagPPO without shield incur the highest costs (53.4 and 49.8 respectively). SAC without shield is notably lower at 21.2 despite having no safety mechanism, suggesting that SAC's entropy-regularized exploration naturally avoids prolonged hazard exposure.

5.2 Effect of the Geometric Shield on Safety

Figure 2 shows average cost per step during training across all conditions. The effect of the shield on training-time cost is inconsistent across algorithms. For PPO and LagPPO the shield produces a small reduction in mean cost, but confidence bands overlap substantially throughout the entire run. For SAC, the shield-on condition shows no improvement over shield-off, and evaluation results show shield-on actually incurring higher mean cost (57.3 vs. 21.2), though with very high variance across seeds (± 56.0 vs. ± 17.0). This high variance makes strong conclusions impossible from three seeds alone. Figure 3 shows the shield intervention rate across all shield-on conditions and it reveals a more structured pattern. PPO shield-on has the highest intervention rate throughout training, rising to approximately 0.003 by 1M steps and remaining clearly separated from all other conditions. LagPPO shield-on is second at approximately 0.002. SAC shield-on and RCPO shield-on are substantially lower at around 0.0005 to 0.001. This ordering suggests that unconstrained algorithms rely most heavily on the shield, while algorithms with explicit cost signals learn to avoid hazards through their objective and require fewer corrections.

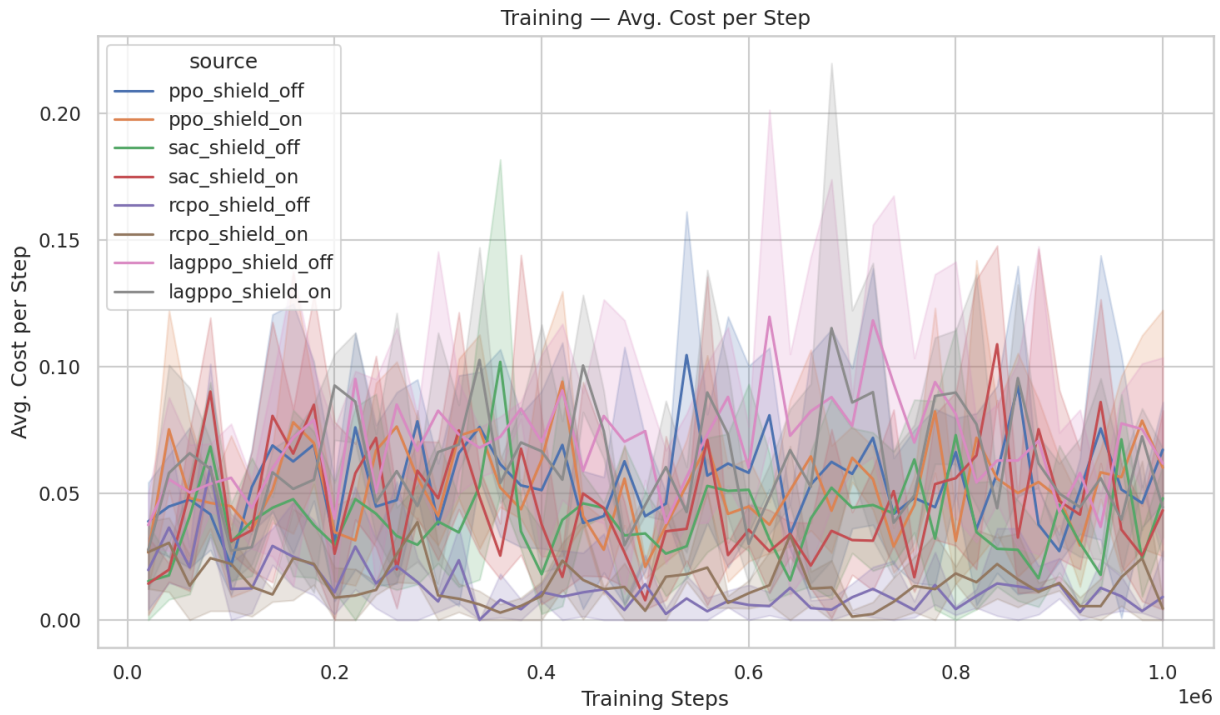

Figure 2: Average cost per step during training (mean ± std across 3 seeds).

5.3 Complementarity of Shielding and Lagrangian Constraints

Figure 4 shows the Lagrange multiplier λ trajectory for both LagPPO conditions throughout training. Both conditions show monotonically increasing λ, indicating that the cost budget $d = 0.05$ is never satisfied within one million steps and the constraint remains binding throughout training. LagPPO shield-off reaches a mean λ of approximately 0.005 at 1M steps, while LagPPO shield-on reaches approximately 0.003. This separation becomes consistent from around step 200k onward, and the confidence bands diverge in the second half of training.

This is the central finding of the paper. The shield reduces the rate at which λ must grow by preventing some violations from reaching the environment and being counted toward the cost signal. The Lagrangian mechanism then needs to apply less pressure to keep the policy near the constraint boundary. The two mechanisms therefore address safety at different timescales: the shield operates instantaneously at the action level, while the Lagrangian operates distributionally over episodes. When combined, they are complementary rather than redundant.

5.4 Tail Risk and CVaR

CVaR cost at $\alpha = 0.1$, measuring the expected cost of the worst 10% of evaluation episodes, is high and variable across all conditions. Values range from 26.1 for RCPO shield-off to 416.0 for SAC shield-on. RCPO is the only algorithm family where CVaR remains below 100 across both shield conditions, while all other conditions exceed 200. This indicates that no algorithm in this study achieves reliable tail safety within one million training steps. Even conditions with moderate average cost can produce occasional high-cost episodes, which is a relevant concern for real deployment scenarios where worst-case behavior matters more than average behavior.

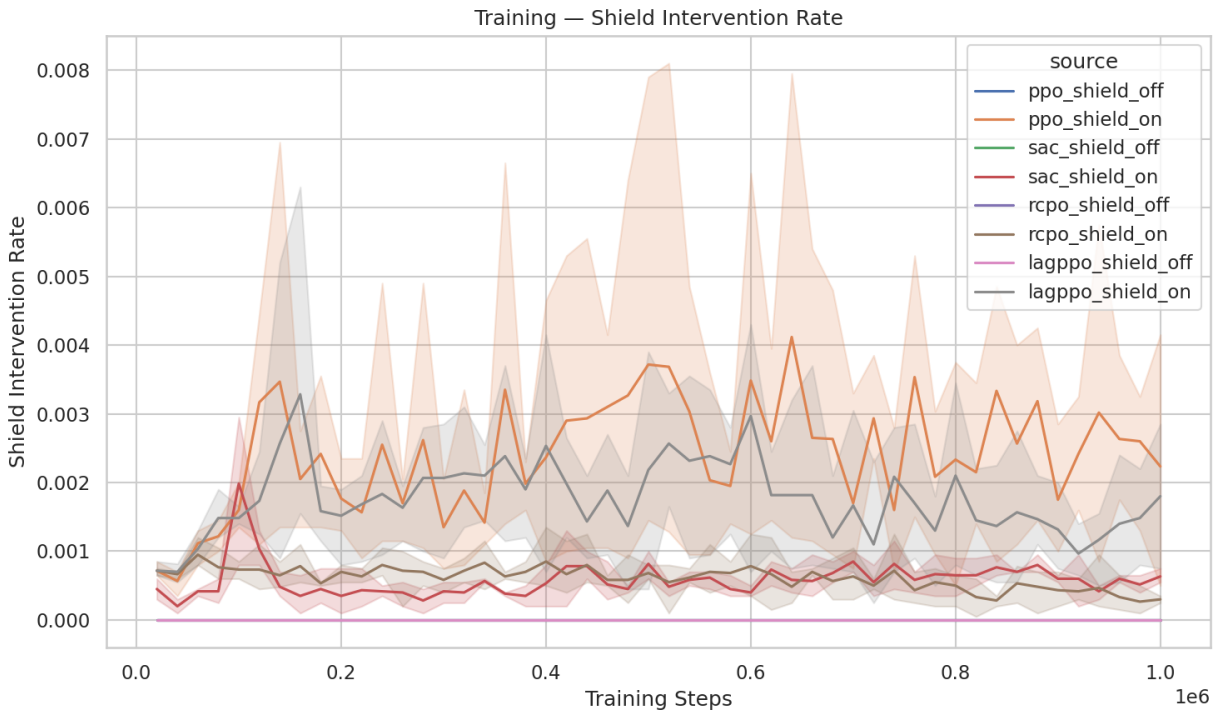

Figure 3: Shield intervention rate during training for shield-on conditions (mean ± std across 3 seeds).

5.5 Seed Variance and Statistical Limitations

A consistent finding across all conditions is high variance across the three seeds. Evaluation costs range from near zero to over 100 within the same algorithm and shield condition: for example, LagPPO shield-on seed 0 achieves average cost of 69.5 while seed 2 achieves 3.1. This variance means that confidence intervals overlap for all pairwise comparisons of return and cost, and no result other than the λ trajectory separation and the RCPO versus rest distinction can be stated with statistical confidence from three seeds. We report these results transparently and discuss implications in Section 6.

6 Discussion

6.1 Safety Mechanisms Operate at Different Timescales

The central finding of this paper is that geometric shielding and Lagrangian constraints address safety at fundamentally different levels of the learning system, and that this difference makes them complementary rather than redundant. The shield acts locally and instantaneously: at each step it prevents specific unsafe actions from reaching the environment, regardless of what the policy has learned so far. The Lagrangian constraint acts globally and gradually: it adjusts the policy's optimization objective in response to accumulated cost over many episodes, slowly shaping the distribution of visited states. Because these two mechanisms operate at different timescales, combining them produces a measurable interaction. The shield reduces the number of violations that accumulate in the cost signal, which in turn reduces the pressure on the Lagrangian mechanism

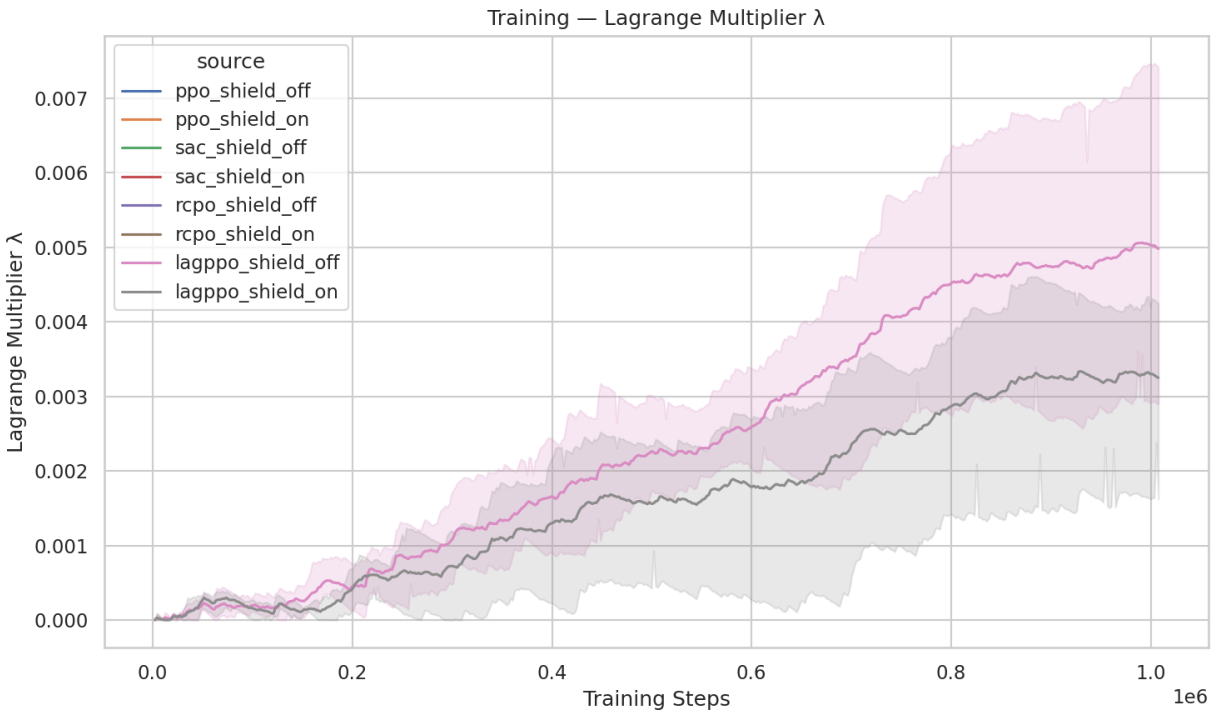

Figure 4: Lagrange multiplier λ trajectory for LagPPO conditions (mean $\pm$ std across 3 seeds).

to grow λ. The result is a lower multiplier trajectory throughout training, visible in Figure 4.
This finding has a practical implication. In safe RL system design, safety mechanisms are often treated as alternatives: one chooses either a shield or a constraint-based algorithm. Our results suggest that this framing misses an important possibility. A shield and a Lagrangian constraint together form a more complete safety architecture, where runtime action correction handles immediate violations and the learning objective handles the policy's long-run behavior. The two mechanisms do not conflict because they operate at different points in the decision pipeline.

6.2 Interpreting Unexpected Results

Two results in this paper were not immediately intuitive and deserve closer examination.

The first is RCPO's behavior. The fixed penalty coefficient suppresses violations effectively, giving RCPO the lowest average cost and CVaR across all conditions. However, it also suppresses task performance almost entirely, with both RCPO conditions returning near-zero or negative episodic return. The policy appears to learn that the safest strategy is to avoid task-seeking behavior altogether. This is a known failure mode of static penalty methods: when the penalty coefficient is too large relative to the task reward, the penalty dominates and the policy finds that inaction or avoidance is locally optimal. It highlights a fundamental limitation of fixed-coefficient approaches: as Tessler et al. [2018] note, selecting a constant penalty coefficient that reliably satisfies constraints without suppressing task performance is non-trivial and domain-dependent, which is precisely the problem that adaptive methods like LagPPO are designed to address.
The second unexpected result is SAC's relatively low cost without any safety mechanism. SAC without a shield achieves mean evaluation cost of 21.2, lower than PPO without a shield at 53.4, despite both algorithms having no cost awareness. One plausible explanation is that SAC's entropy-

regularized objective encourages the policy to maintain diverse, exploratory behavior rather than committing to a fixed trajectory through the environment. A policy that constantly varies its paths is less likely to repeatedly traverse the same hazard regions, which would naturally reduce accumulated cost. This is not a safety mechanism in any meaningful sense, but it suggests that exploration strategy has a non-trivial effect on observed safety metrics even without explicit constraints.

6.3 Limitations and Future Work

Several limitations constrain the conclusions of this paper.

The most significant is the number of seeds. With three seeds per condition, confidence intervals overlap for all pairwise comparisons of return and cost. The λ trajectory separation and the RCPO versus rest distinction are the only results that remain interpretable under this variance. Repeating these experiments with five or more seeds would substantially improve statistical reliability.

A second limitation is the single environment: all results come from SafetyPointPush1-v0, a planar task with static, fully observable hazard geometry. The geometric shield works well in this setting because the environment structure is simple and known in advance. In more complex robotic settings, hazard geometry may be partially observable, dynamic, or unknown at deployment time. Whether the complementary interaction between shielding and Lagrangian constraints generalizes to these settings is an open question that requires evaluation on more diverse benchmarks such as Safety-Gymnasium's robot arm tasks or more complex navigation environments.

A third limitation concerns the shield's assumptions. The shield assumes perfect knowledge of hazard positions and performs a deterministic geometric projection without accounting for dynamics uncertainty or sensor noise. A more realistic shield, for example one based on control barrier functions [Ames et al., 2019] or learned constraint models [Dalal et al., 2018], might produce qualitatively different interactions with the learning algorithms.

Finally, our evaluation focuses primarily on average safety metrics and CVaR over evaluation episodes. Safety in deployment often depends on rare worst-case events that are difficult to characterize from a small number of episodes. A more thorough safety analysis would require substantially more evaluation episodes, formal verification methods, or adversarial testing to probe the boundaries of the learned policies.

Taken together, these limitations suggest a natural direction for future work: a larger-scale factorial study across multiple environments and more seeds, using more realistic safety layers, with evaluation protocols designed to measure tail risk more precisely. The framework released with this paper is designed to support exactly this kind of extension.

7 Conclusion

Safe reinforcement learning requires balancing task performance against safety constraints, yet the field has largely treated safety mechanisms as competing alternatives rather than composable components. In this work we presented a systematic empirical study of four reinforcement learning algorithms, PPO, SAC, RCPO, and LagPPO, each trained with and without a geometric keepout shield on the SafetyPointPush1-v0 benchmark. Our 4×2 factorial design across three random seeds was designed to isolate the effect of shielding within each algorithm family and to measure how safety mechanisms interact when combined.

Our results show that safety mechanisms are not interchangeable and exhibit distinct trade-offs that persist across one million training steps. RCPO achieves the lowest violation rate and CVaR cost but suppresses task performance almost entirely, while unconstrained algorithms achieve competitive return at the cost of high safety violations. No algorithm satisfies the cost budget within the training

horizon, indicating that full constraint satisfaction remains difficult under limited compute. The clearest and most interpretable finding is that geometric shielding and Lagrangian constraints are complementary: adding a shield measurably reduces the rate of Lagrange multiplier growth in LagPPO, providing evidence that the two mechanisms operate at different timescales and address safety at different levels of the system.

More broadly, these results suggest that safe RL should be approached as a system design problem. Rather than asking which safety method is best, the more productive question is how different safety layers interact and how they can be composed to address safety at multiple levels simultaneously. We hope the experimental framework released with this paper supports future work in this direction.

A Hyperparameters and Implementation Details

This appendix provides the full hyperparameter configuration for all experimental conditions, notes on key design decisions, and instructions for reproducing the experiments.

A.1 Hyperparameter Tables

Table 2 lists hyperparameters shared across all conditions. Tables 3, 4, 5, and 6 list algorithm-specific hyperparameters.

Table 2: Shared hyperparameters across all conditions.

Parameter	**Value**
Environment	SafetyPointPush1-v0
Total training steps	1,000,000
Parallel workers	4 (SubprocVecEnv)
Random seeds	0, 1, 2
Policy network	MLP, 2 hidden layers of 64 units, tanh activation
Value network	MLP, 2 hidden layers of 64 units, tanh activation
Optimizer	Adam
Evaluation episodes	20
Evaluation seed	100
Shield applied at eval	No
Framework	Stable-Baselines3

Table 3: PPO hyperparameters (shield-off and shield-on).

Parameter	**Value**
Learning rate	3×10^{-4}
Number of steps (rollout)	2,048
Batch size	64
Number of epochs	10
Clip range ϵ	0.2
Entropy coefficient	0.0
Value function coefficient	0.5
Max gradient norm	0.5
GAE λ	0.95
Discount γ	0.99

Table 4: SAC hyperparameters (shield-off and shield-on).

Parameter	**Value**
Learning rate (actor)	3×10^{-4}
Learning rate (critic)	3×10^{-4}
Replay buffer size	1,000,000
Batch size	256
Soft update τ	0.005
Discount γ	0.99
Entropy coefficient	auto-tuned
Learning starts	1,000
Train frequency	1
Gradient steps	1

Table 5: RCPO hyperparameters (shield-off and shield-on). RCPO uses a PPO backbone with a fixed cost penalty added to the reward at each step.

Parameter	**Value**
PPO backbone hyperparameters	Same as Table 3
Fixed penalty coefficient λ_{fixed}	1.0
Penalty applied as	$r'_t = r_t - \lambda_{\text{fixed}} \cdot c_t$

Table 6: LagPPO hyperparameters (shield-off and shield-on).

Parameter	**Value**
PPO backbone hyperparameters	Same as Table 3
Cost budget d	0.05
Multiplier learning rate η_λ	5×10^{-4}
Multiplier initialization	0.0
Multiplier clipping	$\lambda \geq 0$

A.2 Cost Budget Selection

We set the cost budget to $d = 0.05$. In Safety-Gymnasium environments, the cost signal corresponds to entering hazard regions and is binary at each timestep, so the budget effectively limits the fraction of unsafe steps experienced by the agent. A value of $d = 0.05$ therefore allows occasional violations during exploration while still requiring the policy to actively avoid hazards. Given the episode horizon of the environment, this budget corresponds to only a small number of violations per episode on average, making the constraint meaningful without preventing exploration early in training.

In practice, smaller budgets (e.g., $d = 0.01$) may destabilize training by causing the Lagrange multiplier to grow rapidly before the policy has learned to avoid hazards, while larger budgets (e.g., $d = 0.1$) weaken the constraint and reduce the pressure on the policy to learn safe behavior. The chosen value represents a moderate safety constraint that induces a clear trade-off between task performance and safety, which is the regime of interest for this study.

A.3 Shield Implementation

The geometric shield is implemented as a wrapper around the environment's `step` function. At the start of each episode, the shield reads hazard positions and radii from the environment's `hazards_pos` and `hazards_size` fields. At each step, the shield receives the policy's proposed action a_t and the current agent position x_t, computes the predicted next position as $\hat{x}_{t+1} = x_t + \Delta t \cdot a_t$, and checks whether $\hat{x}_{t+1}$ falls within any hazard region of radius r_h centered at h. If $\|\hat{x}_{t+1} - h\| < r_h$ for any hazard h, the component of a_t directed toward h is removed:

$$a_t^{\text{safe}} = a_t - \left(\frac{a_t \cdot (h - x_t)}{\|h - x_t\|^2} \right) (h - x_t). \tag{5}$$

The corrected action a_t^{safe} is passed to the environment in place of a_t. The shield records a binary intervention flag at each step, which is used to compute the shield intervention rate reported in Section 5.2.

A.4 Reproduction Commands

All experiments can be reproduced using the code released at:
`https://github.com/samuelepesacane/Safe-Reinforcement-Learning-for-Robotic-Manipulation/`
To install dependencies:

```
pip install -r requirements.txt
```

To run a single training condition (example: LagPPO with shield, seed 0):

```
python -m src.train --env_id SafetyPointPush1-v0 \
  --algo lagppo --total_timesteps 1000000 \
  --seed 0 --num_envs 4 --eval_freq 20000 \
  --cost_budget 0.05 --lr_lambda 5e-4 --use_shield \
  --log_dir logs/lagppo_shield_on_seed0 \
  --ckpt_dir checkpoints/lagppo_shield_on/seed_0
```

To run all 24 conditions sequentially:

```
nohup bash run_full_ablation.sh > logs/full_ablation_run.log 2>&1 &
```

To evaluate all trained checkpoints:

```
for algo in ppo sac rcpo lagppo; do
  for condition in shield_off shield_on; do
    for seed in 0 1 2; do
      python -m src.evaluate \
        --env_id SafetyPointPush1-v0 \
        --model_path checkpoints/${algo}_${condition}/seed_${seed}/latest \
        --episodes 20 --seed 100 \
        --log_dir results/eval_${algo}_${condition}_seed${seed}
    done
  done
done
```

To reproduce all plots from training logs:

```
python -m src.visualize \
  --log_dirs logs/ppo_shield_off_seed{0,1,2} \
             logs/ppo_shield_on_seed{0,1,2} \
             logs/sac_shield_off_seed{0,1,2} \
             logs/sac_shield_on_seed{0,1,2} \
             logs/rcpo_shield_off_seed{0,1,2} \
             logs/rcpo_shield_on_seed{0,1,2} \
             logs/lagppo_shield_off_seed{0,1,2} \
             logs/lagppo_shield_on_seed{0,1,2} \
  --out_dir results/final_plots
```

Full environment configuration, dependency versions, and hardware requirements are documented in the repository `README.md`.

References

Altman, E. (1999). *Constrained Markov Decision Processes: Stochastic Modeling*. Routledge.

Ames, A. D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., and Tabuada, P. (2019). Control barrier functions: Theory and applications. In *18th European Control Conference (ECC)*, pp. 3420–3431.

Dalal, G., Dvijotham, K., Vecerik, M., Hester, T., Paduraru, C., and Tassa, Y. (2018). Safe exploration in continuous action spaces. *arXiv:1801.08757*.

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML 2018*. arXiv:1801.01290.

Ji, J., Zhang, B., Zhou, J., Pan, X., Huang, W., Sun, R., Geng, Y., Zhong, Y., Dai, J., and Yang, Y. (2023a). Safety-gymnasium: A unified safe reinforcement learning benchmark. *arXiv:2310.12567*.

Ji, J., Zhou, J., Zhang, B., Dai, J., Pan, X., Sun, R., Huang, W., Geng, Y., Liu, M., and Yang, Y. (2023b). Omnisafe: An infrastructure for accelerating safe reinforcement learning research. *arXiv:2305.09304*.

Ray, A., Achiam, J., and Amodei, D. (2019). Benchmarking safe exploration in deep reinforcement learning. *OpenAI Technical Report*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

Tessler, C., Mankowitz, D. J., and Mannor, S. (2018). Reward constrained policy optimization. *arXiv:1805.11074*.